

UT1 — Pràctica de repàs: TechShop

CIFP Pau Casesnoves · IFC33C · Curs 2026–2027

Mòdul: 0613. Desenvolupament web en entorn servidor

Professor: David Pons

Objectiu

Implementar el backend de demostració de TechShop, una botiga de productes tecnològics. L'aplicació ha de permetre simular comandes amb dos mètodes de pagament, preparar carrets independents, enviar notificacions i registrar les operacions.

Punt de partida

Crea un projecte Java amb Spring Boot, Maven i Spring Web. Organitza les classes segons la seva responsabilitat: controladors, serveis i model. Els controladors reben les peticions i deleguen les operacions als serveis.

Pots simular els pagaments, el correu i l'accés a fitxers amb cadenes de text i missatges de consola. Els productes es poden representar amb un `String`. No cal base de dades, formularis ni connexions amb serveis externs.

Spring ha de gestionar els components i proporcionar-los les dependències. Determina les anotacions necessàries a partir dels comportaments descrits. Les respostes HTTP poden ser textuals, però han de mostrar les dades indicades per poder fer les comprovacions.

Requisits

1. Processadors de pagament

Defineix la interfície `PaymentProcessor` i dues implementacions: `CreditCardProcessor` i `PaypalProcessor`. Totes dues han de poder processar un import i retornar un resultat que identifiqui el mètode de pagament utilitzat.

- Les dues implementacions han d'estar disponibles al mateix temps.
- Una dependència de tipus `PaymentProcessor` sense preferència concreta ha de rebre el processador de targeta de crèdit.
- La variant de servei que gestioni pagaments amb PayPal ha de rebre expressament aquest processador, mantenint `PaymentProcessor` com a tipus de la dependència.

2. Comandes i descomptes opcionals

Implementa `OrderService` i organitza les variants necessàries per als dos mètodes de pagament. Les dependències imprescindibles s'han de rebre per constructor.

Algunes instal·lacions disposen de `DiscountService`, que aplica un descompte del 10 % a l'import rebut. Aquest servei ha de ser una col·laboració opcional rebuda per setter.

- Amb `DiscountService` registrat, una comanda de 100 euros s'ha de pagar per 90 euros.
- Sense aquest component, l'aplicació ha d'arrencar i la mateixa comanda s'ha de pagar per 100 euros.
- La resposta ha de mostrar el processador utilitzat, l'import original i l'import final.
- Cada comanda ha de deixar una entrada a `LogManager` amb el processador utilitzat i l'import final.

Processar una comanda no ha d'enviar cap correu. Les notificacions es demanen amb una operació independent.

3. Un carret nou per operació

Implementa `ShoppingCart` amb un identificador únic i un `ArrayList` de productes inicialment buit. Cada vegada que es demani un carret al contenidor, s'ha d'obtenir una instància nova. Dues operacions del mateix usuari també han d'obtenir carrets diferents.

Centralitza aquesta petició en `ShoppingCartFactory`, amb un mètode `createCart()`. La fàbrica ha d'utilitzar `ObjectProvider<ShoppingCart>` per obtenir el carret de Spring en el moment que es necessita.

`ShoppingCartService` és un servei compartit per tota l'aplicació. Rep la fàbrica per constructor i la utilitza en cada operació; no conserva un únic carret injectat al seu constructor. Ni el servei ni la fàbrica han de construir els carrets manualment.

Prepara aquestes dues operacions:

- Crear un carret, afegir-hi el producte rebut i retornar-ne l'identificador i la llista de productes.
- Crear dos carrets dins una mateixa petició HTTP: afegir `Teclat` al primer i `Ratolí` al segon, i retornar les dades de tots dos. Els identificadors han de ser diferents i cada llista ha de contenir només el seu producte.

4. Notificacions per correu

Implementa `EmailSender`, que prepara una connexió costosa amb el servidor SMTP quan es crea. Simula aquesta preparació amb un missatge al constructor. El mètode `send()` ha de simular l'enviament amb un altre missatge.

`NotificationService` és compartit, es crea a l'arrencada i rep `EmailSender` pel constructor.

- Arrencar l'aplicació, preparar carrets i processar comandes no ha d'inicialitzar `EmailSender`.
- La connexió SMTP només s'ha de preparar en demanar la primera notificació.
- Els enviaments següents han de reutilitzar el mateix component sense repetir-ne la preparació.

5. Registre i gestió dels recursos

Implementa `LogManager` com un component compartit que gestiona un recurs propi per registrar les comandes.

- Una vegada construït i amb les dependències ja proporcionades, prepara el recurs abans de començar a prestar servei. Pots simular-ho amb un missatge d'obertura.
- En registrar una comanda, mostra el processador i l'import final.
- Quan el context de Spring es tanqui ordenadament, allibera el recurs i mostra un missatge de tancament.

L'obertura i el tancament es fan una vegada durant el funcionament de l'aplicació; no s'han de repetir per cada comanda.

Rutes HTTP de l'aplicació

Implementa els controladors necessaris amb aquestes rutes. En totes les peticions de prova s'utilitzen imports positius i productes no buits.

Petició	Comportament
`GET /shop/order/card?amount=100`	Processa una comanda amb el processador per defecte. Retorna el processador i els imports original i final. Registra la comanda.
`GET /shop/order/paypal?amount=100`	Fa la mateixa operació amb el processador de PayPal seleccionat expressament.
`GET /shop/cart?product=Teclat`	Obté un carret nou, hi afegeix el producte i retorna l'identificador del carret i els productes.
`GET /shop/cart/pair`	Obté dos carrets nous en la mateixa petició i mostra els identificadors i els productes de cadascun segons l'apartat 3.
`GET /shop/notification`	Demana un enviament a `NotificationService` i retorna una confirmació.

Els paràmetres `amount` i `product` han d'arribar a l'operació del servei. Utilitza les dependències injectades per seleccionar el processador de pagament; no creïs els processadors dins els mètodes dels controladors.

Proves de verificació

Pots executar les peticions des del navegador o amb un fitxer `.http`. Pren com a base `http://localhost:8080` i recull les respostes o els missatges necessaris per comprovar aquests resultats:

1. **Arrencada.** Inicia l'aplicació. Ha d'aparèixer una única obertura del recurs de registre i cap preparació de la connexió SMTP.
2. **Selecció del pagament.** Amb `DiscountService` disponible, crida `/shop/order/card?amount=100` i `/shop/order/paypal?amount=100`. Han de mostrar processadors diferents, import original 100 i import final 90.
3. **Carrets entre peticions.** Crida dues vegades `/shop/cart?product=Teclat`. Els identificadors dels carrets han de ser diferents i cadascun ha de contenir una única entrada `Teclat`.
4. **Dos carrets en una petició.** Crida `/shop/cart/pair`. Els dos carrets han de tenir identificadors diferents: el primer ha de contenir només `Teclat` i el segon només `Ratolí`. Repeteix la petició: els identificadors nous han de ser diferents dels anteriors.
5. **Primer enviament de correu.** Després de les proves de comandes i carrets, encara no s'ha d'haver preparat SMTP. Crida dues vegades `/shop/notification`: només el primer enviament ha de mostrar la preparació; tots dos han de mostrar l'enviament.
6. **Descompte absent.** Retira el registre de `DiscountService` al contenidor i reinicia. L'aplicació ha d'arrencar. Repeteix les dues comandes amb import 100: ara l'import final ha de ser 100 en totes dues.
7. **Registre i tancament ordenat.** Comprova que cada comanda ha deixat un missatge amb el processador i l'import final. Atura l'aplicació de manera ordenada i comprova el missatge d'alliberament del recurs. L'obertura i el tancament no s'han de repetir per cada comanda.

Evidències del treball

Prepara el projecte executable i un recull breu de les respostes i els missatges de les proves. Identifica cada evidència amb el número de la comprovació corresponent.